

Scalability and Throughput

The current microservices approach (separate “embedding” and “ETL” containers) is a good start, but scaling will require care. Each service should remain **stateless** so you can run many replicas behind a load balancer. For example, the embedding API running on Uvicorn could be scaled horizontally (multiple containers) to handle more simultaneous requests. However, loading a heavy ML model in each container is costly – use a single shared instance per container (see *Performance* below) or run multiple workers with a preload strategy ¹ ².

For large data (41M+ tokens), batch processing and parallelization become critical. Instead of sequentially embedding each document, consider distributed processing frameworks. For example, one can chunk input data and parallelize embedding with tools like [Ray](#) to use multiple CPUs/GPUs simultaneously ³. Similarly, conversion to efficient on-disk formats (e.g. Parquet) can reduce I/O overhead for huge text corpora ⁴.

On the Weaviate side, **vector index scaling** must be planned. Weaviate suggests roughly *6 GB of RAM per 1 million, 1024-dimensional vectors* ⁵. For 41M tokens, even with medium-dimensional embeddings (e.g. 384-dim), this implies tens of GB of RAM. To manage this: use a lower-dimensional or quantized embedding model (e.g. 256-d vectors or enable product quantization) to cut memory (e.g. Weaviate estimates ~1.5 GB per million for 256-d without quantization ⁵). Also consider multi-tenancy: if your Pali, English, Burmese, etc. data share the same schema, store them as different **tenants** in one collection. Weaviate’s multi-tenancy mode lets you isolate logically separated datasets without creating separate collections for each language ⁶. This reduces overhead compared to many collections. If needed, you can use a “dynamic” index type (flat-to-HNSW) or enable quantization to trade off memory vs performance ⁷ ⁸.

Table: Key Scalability Issues & Suggestions

Aspect	Issue / Limitation	Suggestion / Best Practice
Embedding Inference	Model loaded per container/request; single worker only	Load model once at startup (global/app state) ² ⁹ ; use Gunicorn + <code>--preload</code> to share model across workers ¹ . Cache embeddings if possible.
ETL Throughput	Sequential ingestion of millions of records (slow)	Batch and parallelize vectorization (e.g. use Ray or multiprocessing) ³ . Process data formats (JSON→Parquet) to speed I/O ⁴ .
Weaviate Indexing	Large vector count; high RAM needed (esp. 1024-dim)	Use lower-dim or quantized embeddings (6 GB→2 GB per million by quantization) ⁵ . Use multi-tenancy to combine subsets under one schema ⁶ .

Maintainability and Code Organization

The code and Dockerfiles should be organized cleanly to help new developers. For example, keep related Python code and configuration in well-structured directories (e.g. an `app/` or `src/` folder,

separate modules for API vs ETL logic). Provide a clear README or doc explaining how services interact. Use consistent style (PEP8, docstrings) and add comments for non-obvious logic.

Dependencies should be **explicitly versioned**. The current `requirements.txt` has no version pins; this makes builds non-reproducible and risky (upgrades can silently break things). Pin exact versions for each package to ensure consistent environments ¹⁰. For example:

```
fastapi==0.113.1
uvicorn==0.23.1
weaviate-client==4.4.1
sentence-transformers==2.2.2
```

and so on. You might also split dev/test dependencies into a separate file (e.g. `requirements-dev.txt`) ¹¹ so that only production deps go into your image. Regularly audit dependencies for security (e.g. using `pip-audit` ¹²) and remove unused libraries to keep the codebase minimal.

Finally, avoid copying code redundantly. For instance, the ETL Dockerfile appears to mix the embedding server and pipeline logic. These should be separate services or properly modularized. Ensure each container builds *only the code it needs*. In production, use `COPY` commands (or a `setup.py` / `pyproject.toml`) to install the actual ETL application code into the container, not just bind-mount volumes. A clean directory structure with clear entrypoints makes onboarding new developers much easier.

Performance and Inference Efficiency

A big performance issue can come from **how the embedding model is loaded**. If the code loads the SentenceTransformer inside the request handler, then every call will re-load the model, causing huge latency. Instead, load the model **once** on startup (e.g. in global scope or a FastAPI startup event), so all requests reuse it ² ⁹. FastAPI allows putting long-lived objects on `app.state` so they persist between calls ⁹. The advice is clear: *“Load your model into memory just once when the application starts, rather than loading it with each request”* ².

If you want multiple worker processes (to utilize multiple CPU cores), consider using Gunicorn with Uvicorn workers and the `--preload` option ¹. This loads the application (and its model) once in the master process and then forks workers, saving memory and speeding startup. Without preload, each worker would load its own copy of the model, multiplying memory use.

For the embedding API's throughput, note that the Dockerfile fixed `--workers 1`. If higher concurrency is needed, increase workers or run multiple containers (behind a load balancer). However, remember each worker/container will need the model loaded. If GPU acceleration is available, you might leverage it for embedding (though managing GPUs in Docker adds complexity).

Finally, consider caching or batching at a higher level: if the same query text is requested often, cache its embedding for a time. Also use asynchronous I/O for any DB or network calls (FastAPI and Uvicorn support async endpoints) so that one slow I/O doesn't block others. In short, ensure the service fully reuses the loaded model and handles concurrent requests via asynchronous code or multiple workers.

Docker Image Optimization

The ETL image is alarmingly large (~8 GB) mainly because of heavy ML dependencies. Indeed, installing `sentence-transformers` pulls in PyTorch (with CUDA/NVIDIA libs by default) which can be multi-gigabytes. Analysis shows much of the image size is `/usr/local/lib/python3.11/site-packages/torch` and related `nvidia` libraries ¹³.

Recommendations:

- **Use CPU-only PyTorch.** Instead of letting pip pull GPU-enabled PyTorch, explicitly install the CPU wheels: e.g.

```
RUN pip install --no-cache-dir torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cpu
RUN pip install --no-cache-dir sentence-transformers
```

This cuts out ~6 GB of CUDA tooling ¹⁴. The linked solution shows such a change yields a ~1.4 GB image instead of ~8 GB ¹⁴.

- **Remove build tools after use.** The ETL Dockerfile installs `build-essential`. This is only needed for compiling wheels (e.g. sentencepiece). After `pip install`, purge dev packages to shrink the image. For example:

```
RUN apt-get update && \
    apt-get install -y --no-install-recommends build-essential curl && \
    pip install --no-cache-dir -r requirements.txt && \
    apt-get purge -y build-essential && \
    rm -rf /var/lib/apt/lists/*
```

This removes compilers and cleans apt caches in one layer.

- **Use multi-stage builds if appropriate.** For very large builds, one could install in a “builder” stage and copy only needed libs into a slimmer final image. In practice for pure Python this yields limited gain, but it *can* help if you need tools that you then delete.
- **Fix Dockerfile commands.** Only the last `CMD` in a Dockerfile takes effect. The ETL Dockerfile has two `CMD`s (one for Uvicorn and another for the pipeline script). This should be cleaned up. Decide whether the ETL container is running a web app or a one-off script. If it’s just running `pipeline.py`, remove the unused Uvicorn `CMD`. If it needs Uvicorn, swap `CMD/ENTRYPOINT` properly.
- **Use `.dockerignore`.** Exclude local files (e.g. data files, `.git`) from the build context to avoid bloating the image. Copy only what’s needed (as shown above).

Finally, **base image choice:** `python:3.11-slim` is good, but ensure you use the slim *-bullseye* or newer variant. Alpine Python images are smaller but can cause compatibility headaches with PyTorch and sentencepiece. Staying on Debian-slim and removing extras is safer for ML workloads.

Dependency Hygiene

Current `requirements.txt` is minimal but unversioned. This is risky long-term. Best practices include:

- **Pin package versions** ¹⁰. This ensures that rebuilding the Docker image uses the same library versions that you tested with. Without pins, a new release of a dependency could introduce breaking changes.
- **Remove unused dependencies.** For example, if the ETL pipeline never uses `fastapi` or `uvicorn` (since those may belong only to the embedding service), split the requirements accordingly. Each microservice should list only what it needs.
- **Audit and update regularly.** Use tools like `pip-audit` or dependabot to catch vulnerabilities or outdated libraries ¹².
- **Use virtual environments (locally).** For development, use a venv or tools like Poetry/venv to isolate dependencies and test installs before updating the `requirements.txt`. Although Docker images are isolated, keeping your development environment clean prevents “works on my machine” issues.

Model Reuse and Inference Best Practices

To support multiple languages (Pali, English, etc.), use an embedding model that already handles those languages. The [Sentence-Transformers model zoo](#) lists multilingual models (e.g. `paraphrase-multilingual-MiniLM-L12-v2`) that support 50+ languages (including Chinese, Russian, Burmese/Myanmar) ¹⁵. Using one robust multilingual model avoids the complexity of managing a separate model for each language. If your use case evolves, you can switch models more easily.

Reuse the model intelligently: If different services or scripts need embeddings, either call a central embedding API or share a cached model. For example, the ETL pipeline could fetch embeddings by calling the embedding microservice (keeping all model code in one place), or it could load a cached sentence-transformer model from disk once. But it **must not** load a fresh model for each chunk of text. The rule from ML deployment is clear: *“Load your model into memory just once when the application starts...”* ².

For inference speed, batch inputs when possible (many libraries can encode a list of sentences faster than one-by-one). Also, serve the model with optimizations like Torch’s parallelism flags or by exporting to ONNX if extremely high throughput is needed (advanced topics, but worth exploring as you grow). Always measure latency: even with a cached model, larger models (e.g. 1024-dim) will be slower than smaller ones; pick the smallest model that meets accuracy needs.

Summary of Findings

Category	Issue/Risk	Recommendation (with Sources)
Model Loading	If the service loads the SentenceTransformer for every request, latency is huge.	Load model at startup (cache on <code>app.state</code>) ² ⁹ ; use Gunicorn <code>--preload</code> for multiple workers ¹ .
Microservices	ETL and embedding containers overlap (two CMDs); unclear separation of concerns.	Clearly separate services or use one microservice for embedding, one for ETL. Remove extra CMDs. Document each service.
Image Size	ETL image ~8 GB (mostly GPU PyTorch libs) ¹³ .	Use CPU-only PyTorch (<code>--index-url</code> CPU) ¹⁴ ; remove build tools; combine layers.
Dependencies	<code>requirements.txt</code> has unpinned versions and possibly unnecessary libs.	Pin each version ¹⁰ ; split requirements per service (prod vs dev); audit regularly ¹² .
Scalability (Data)	41M+ tokens → millions of vectors; default HNSW indexing is memory-hungry.	Use lower-dim or quantized embeddings; consider Weaviate multi-tenancy and dynamic indexes ⁶ ⁵ . Parallelize ETL (batching/Ray) ³ .
Maintainability	Code mostly in single files; newcomers may be confused by structure.	Modularize code (use packages, configs); add README/docs; follow style guides. Encourage virtualenvs and pip-tools.

By adopting these improvements – e.g. pinning versions, cleaning the Docker build, loading models once, and planning Weaviate capacity – the system will become more robust, easier to maintain, and ready to grow. For junior developers, emphasize clear project structure (for example, a folder per service with its own `requirements.txt`), meaningful docstrings, and simple examples of API calls. Over time, also consider adding unit tests and CI checks to catch issues early.

Sources: Best practices and experiences from FastAPI, Docker, and ML deployment guides ² ¹³ ⁵ ¹⁰ , and related community Q&A ¹ ⁹ ³ .

¹ deployment - loading models in FastAPI projects at startup - Stack Overflow
<https://stackoverflow.com/questions/65636962/loading-models-in-fastapi-projects-at-startup>

² FastAPI: The Modern Toolkit for Machine Learning Deployment | by Reza Shokrzad | Medium
<https://medium.com/@reza.shokrzad/fastapi-the-modern-toolkit-for-machine-learning-deployment-af31d72b6589>

³ ⁴ Building a Scalable RAG Data Ingestion Pipeline for Large-Scale ML Workloads | by Pronnoy Goswami | Medium
<https://medium.com/@pronnoy.goswami/building-a-scalable-rag-data-ingestion-pipeline-for-large-scale-ml-workloads-f6e05d4f8982>

⁵ ⁶ ⁷ ⁸ Best practices | Weaviate Documentation
<https://docs.weaviate.io/weaviate/best-practices>

9 python - How to initialize a global object or variable and reuse it in every FastAPI endpoint? - Stack Overflow

<https://stackoverflow.com/questions/76322463/how-to-initialize-a-global-object-or-variable-and-reuse-it-in-every-fastapi-endp>

10 11 12 How to Auto-Generate requirements.txt for Python Projects: Complete Guide to pipreqs and pigar - DEV Community

<https://dev.to/livingdevops/how-to-auto-generate-requirementstxt-for-python-projects-complete-guide-to-pipreqs-and-pigar-19p0>

13 14 python - How do I slim down SBERT's sentencer-transformer library? - Stack Overflow

<https://stackoverflow.com/questions/77205123/how-do-i-slim-down-sberts-sentencer-transformer-library>

15 Pretrained Models — Sentence Transformers documentation

https://www.sbert.net/docs/sentence_transformer/pretrained_models.html